
COMPUTATIONAL WORKFLOWS REPORT

SUMMER SEMESTER 2025

Francesco Barlow

Johanna Engeln

October 24, 2025

Development of the RNAseq pipeline rnaseqy

1 Introduction

This report describes the development of a pipeline built with nf-core [1] that processes transcriptomic data and performs the initial steps of a typical RNA-seq analysis, including quality control, read trimming, alignment to a reference genome, marking of duplicate reads and generation of a sample against gene count table. The pipeline was named rnaseqy.

1.1 Pipelines

Pipelines, in computer science, are sets of processes connected in series that operate on data. They are especially relevant in bioinformatics, where experimental raw data must be repeatedly pre-processed and analysed. A pipeline typically takes some data as input, runs successive tasks that refine it step by step, and finally produces a meaningful output. The reason pipelines are used is to automate complex analyses, reducing human error in the intermediate steps and ensuring reproducible results.

1.2 FAIR principles and reproducibility

The FAIR principles (Findable, Accessible, Interoperable and Reusable) describe how scientific data and software should be managed to support transparency and reuse [2]. In bioinformatics, applying such principles is essential because analyses are often complex and must be shareable across different groups and institutes; by sharing common guidelines the main sources of bias and differentiation are removed. Reproducibility is closely related to FAIR and means that the same analysis can be repeated and the same results are obtained in each run. Containers make this possible by standardizing how tools are executed and creating the same exact environment for pipelines to run from. These ideas represent the base on which nf-core is built, and this project follows the same approach to ensure the resulting pipeline is clear, shareable and reproducible.[2]

1.3 nf-core

nf-core [3] is a community effort to collect a curated set of analysis pipelines built with Nextflow [4]. nf-core builds on Nextflow and provides the framework (standardized structures, containerization), while Nextflow provides the underlying engine (scalability, portability). The main objective behind the nf-core project is to provide pipelines that follow strict guidelines and with clear examples so that each pipeline can be very easily reproduced on any device and the results remain unchanged.

1.4 RNA-seq

RNA-seq is a Next Generation Sequencing (NGS) technique used to identify and quantify RNA molecules given the snapshot of a transcriptome at a specific time. In RNA-seq, cDNA derived from RNA is aligned to a reference genome and quantified using a gene annotation file. The main use application for RNA-seq is to measure and compare gene expression among samples in different time points or conditions. The process starts from raw sequencing reads, usually

in FASTQ format, that are computationally analysed to become meaningful biological data as read counts. There are several intermediate steps in this analysis, from quality control to trimming and alignment, and for this reason a pipeline that includes all computations in an orderly manner is an optimal way of approaching the task. [5]

2 Methods

2.1 Structure of a nf-core pipeline

A nf-core pipeline follows a clear principal how it is organized in there folder and file structure. When using the command `nf-core pipelines create` all the necessary files and folders are automatically generated. During the process of generating the following files and folders were important for our work:

main.nf This is the main file and is executed when running the pipeline. In this file the pipeline is initialized and it calls the workflow we created in the `workflows/` folder. Finally it completes the workflow as well. For our process we did not touch this file at all, but it is an important one to run the pipeline.

nextflow.config This is the main configuration file for the pipeline. All parameters that can be given to the pipeline are added here and have a default value, if it is not specified that they are required. Further it contains all the profiles possible to choose from to select specific settings depending on preferred running environment (e.g the container software). We used the file to add our own parameters, add a profile to fit our computer resources and added some specifications for two processes, to make our pipeline run smoothly on windows and macOS.

README.md The README is a file that includes the basic information and the usage of our pipeline.

nextflow_schema.json This file is used for the input parameter specification. The file was adapted to our parameters at the end of our project. To adapt it easily we used the command `nf-core pipelines schema build` and then the web interface to adapt it for our input parameters.

assets/ This folder holds all additional files for the pipeline. For us the sample sheet was important to modify for the test run and the `input_schema.json` to specify the layout and properties for the input file.

bin/ The `bin/` folder is mentioned by the nf-core documentation of the pipeline file structure, but was the only one we created ourselves. In the folder python scripts are included to run local modules.

conf/ In this folder all other configuration files, next to the main one are stored. For us the `modules.config` and the `test.config` files were the ones we worked with the most.

modules/ It contains reusable workflows in `nf-core/` which we downloaded to use in our pipeline. Further it stores some local created modules.

workflows/ Our main workflow `rnaseq.nf` is stored in this folder.

Furthermore, the file structure of a pipeline includes more files (`CHANGELOG.md`, `CITATIONS.md`, `CODE_OF_CONDUCT.md`, `LICENSE`, `modules.json`, `nf-test.config`, `.gitattributes`, `.gitignore`, `.gitpod.yml`, `.prettierrc`, `.nf-core.yml`, `ro-crate-metadata.json`) which were used in the way they were set up in the default creation. [6]

2.2 Pipeline modules

To build our own RNAseq pipeline we used common nf-core modules or created pipeline specific modules. All modules can be found in the folder `modules/`. To insert one of the currently 1644 available nf-core modules into our pipeline, we used the command `nf-core modules install <tool>`. [7] Our pipeline includes the following common nf-core modules:

Trim Galore! To apply quality and adapter trimming of FastQ files this module is used. It is a wrapper tool around Cutadapt and FastQC. We used trim galore version 0.6.10 which uses cutadapt version 4.9 for removal of the adapter sequences and low quality bases and pigz version 2.8 for parallel handling of the FastQ files. [8]

FastQC This modules runs a FastQC on the trimmed reads in our pipeline and gives a general quality metric about the reads. We used version 0.12.1 [9]

STAR genomegenerate This module belongs to STAR which is a software package used to align DNA sequences to a reference genome. The module STAR genomegenerate creates an index file for STAR taking a Fasta file and a GTF file of the reference genome. We used the STAR version 2.7.11b including samtools version 1.21 for reference file handling and gawk version 5.1.0 for command-line text processing. [10]

STAR align To actually align the reads to the reference genome with STAR this module is used. It takes the trimmed FastQ files, the generated STAR index file and the GTF file of the reference genome as input for the alignment. It further can take a boolean to ignore the annotation GTF file, the sequence platform and the sequence center. In our pipeline the boolean is set to false and the other two inputs are empty. For this module we also used the STAR version 2.7.11b including SAMtools version 1.21 for reference file handling and gawk version 5.1.0 for command-line text processing. [11]

Custom getchromsize This module was needed to create a Fasta index file in a .fai format. In general this module is used to generate a FASTA index file and the file of chromosome sizes. In this pipeline the getchromsizes version 1.22.1 is used. [12]

SAMtools sort The SAMtools in general are a set of utilities to handle DNA sequences. The SAMtools sort module sorts SAM, BAM and CRAM files. In this pipeline it sorts the output BAM file of STAR align. We used SAMtools version 1.22.1. [13]

Picard MarkDuplicates To locate and tag duplicate reads in our aligned sequences the module Picard MarkDuplicates is used. For this module we needed the sorted BAM file, the Fasta index file in the format .fai and the Fasta file of the reference genome. The Picard version 3.4.0 is used. [14]

Subread FeatureCounts This module is used to count mapped reads for genomic features. In the pipeline it is used as one possible output file. For each sample there is one file in the folder `outDir/subread/`. We used subread version 2.0.6. [15]

StringTie StringTie StringTie assembles RNA-Seq alignments into potential transcripts and performs quantification. The output of StringTie can be used to identify differentially expressed genes between samples by using another tool at the end. [16]. In the pipeline we use for this modules the StringTie version 2.2.3. [17]

StringTie Merge With the StringTie Merge the output of the StringTie files is merged together with the annotation GTF file into one summary GTF file. For this modules we used StringTie version 2.2.1. [18]

Further, we included two personal developed local modules which are stored in the folder `local/`. To create the modules we used the command `nf-core modules create`. The command prompts the relevant questions to create all necessary module files and creates the module directly in the right place when using the command in the pipeline directory. [19] The `main.nf` was adapted and includes the code for the pipeline.

One module is **UNZIPPER** which was introduced in our pipeline to handle difficulties with zipped files. To ensure reproducibility in this rather simple module we used the same container as in the module Trim Galore! and set the version to 1.0.0. In the module, `pigz` is used just as in Trim Galore! to have faster and parallel handling of the decompression of zipped files. The `pigz` version used is 2.8.

The second module implemented is **MERGY** as a final module to handle the output. The module takes the gene abundance files from StringTie and merges them in one `merged_counts.tsv` including the gene abundance TPM value (transcripts per million) for each sample in a gene per sample table. To have a quick overview over the samples, the module computes a Principal Component Analysis and plots PC1 against PC2. For this module we created a container via the `sequera` platform and included it in our module to ensure reproducibility. The module uses a python script (python 3.14.0) which is stored in the `bin/`. For the computation, the packages `matplotlib` (version 3.10.7), `pandas` (version 2.3.3) and `scikit-learn` (version 1.7.2) are used.

3 Results

A pipeline is developed by including modules inside the workflow both taken from the nf-core community and personally written for small, specific tasks such as unzipping some files, merging RNA transcript counts for different samples and PCA plotting. The steps in the pipeline can be seen in the following metro map.



Figure 1: Metro map of modules in the pipeline rnaseqy

The pipeline includes both modules present as part of nf-core as well as two newly implemented modules, UNZIPPER and MERGY, as previously presented. [2.2](#)

The necessary inputs are listed below.

- Samplesheet: csv table containing information such as sample name, fastq files and strandedness. The strandedness is necessary for the modules Subread FeatureCounts and StringTie.
- Genome reference file: reference genome of the species in fasta format.
- GFF file: General Feature Format file, tab delimited file that describes the features of DNA/RNA such as genes, exons and coding sequences.
- GTF file: Gene Transfer Format file, tab delimited file that stores genomic annotation data.
- Output directory: path in which output files and folders are saved in

For the current version of the pipeline, the GTF and the GFF file must be passed as zipped files. A test profile is available to run to quickly try the pipeline with preset data. The test profile can be run using the profile test with the command:

```
nextflow run rnaseqy -profile test
```

The outputs of the pipeline are available in the directory set while calling the pipeline. For the test profile this folder is called out/. They include intermediate outputs in a substeps/ folder relative to the used modules as well as usable final outputs for StringTie and Subread FeatureCounts. The most important file is merged_counts.tsv which is a tab separated file containing genes as rows and samples as columns, each data point being the TPM amount of a gene in a specific sample. A PCA plot of the samples' gene expression is also provided, which is useful for identifying and

filtering outliers. Typically, if most samples cluster together while a few diverge, those diverging samples are likely due to error. Removing them can lead to more accurate results. These outputs are generated by the module **MERGY**, written by the authors.

Another important output is the MultiQC report, it contains a summary of the quality control but we modified it to include also important statistics from the alignment step, the final PCA plot and count table. To have the pipeline working according to the FAIR principles, the MultiQC report also includes all versions of the modules used. In this way, we have a unique report which contains all necessary information which can be glanced upon quickly to get an all-round view of pipeline results without needing to deep-dive into single files.

4 Discussion

In this section we discuss the strengths of the pipeline as well as possible improvements and future outlook possibilities.

4.1 Reproducibility in nf-core

By default, creating an nf-core pipeline and following its template and standards makes sure results are reproducible. First of all, all pipelines must use containers to bundle all software dependencies in one single, portable image that makes sure it will run on any machine. Secondly, previous versions of a pipeline can be ran using the `-r` flag and as such we can be sure the exact same code is being run. This is not an issue with our pipeline as only one version exists but could become essential in case there are any future releases. Reproducibility is finally also achieved by the standard template which is provided when running `nf-core pipelines create`. As long as developers follow the guidelines and don't diverge from the base structure, all pipelines will be structured in the same way and thus easily understandable.

4.2 FAIR pipelines in nf-core

The **FAIR** principles were introduced previously and are a focal point of nf-core. Pipelines developed in nf-core are **FAIR** by nature:

- **Findable**, pipelines are centrally available and discoverable on the nf-core website and they each have a unique name
- **Accessible**, all code is open source and publicly available on GitHub, as well as the containers and documentation
- **Interoperable**, containerization makes the pipeline interoperable across different systems and the standardized parameters mean users can switch between pipelines easily
- **Reusable**, the MIT license allows anyone to copy and modify code, and the modular design means these reusable components can be shared between pipelines

For our pipeline `rnaseq` we tried to be as FAIR as possible in the steps we did ourselves, by tracking all the versions used in the modules and working with containers in our own developed modules.

4.3 Improvements

Although the pipeline's scope was limited and the set objectives were achieved, some improvements can be underlined. The pipeline requires the input of two very similar annotation files, **GFF** and **GTF**, which could be avoided with some extra data management inside the pipeline. A possible solution could be to write a custom module that given one of the two files is able to generate the second without requesting it as a user input. The necessity for both files arises due to different modules needing different inputs, and in this case we included modules which need different annotation files. Another option could be the adaptation of the workflow to make it so only a species identifier has to be given in input and all the necessary files are fetched within the run. We explored this option and try to make it work but with the limited time we were not able to get all necessary data in a consistent way.

On the opposite side of the coin, the current inputs could be extended to allow further downstream analysis. In the current version, the only metadata included for each sample in the samplesheet is the strandedness but an improvement could be the inclusion of more information such as condition, replicate or timepoint for further comparisons after gene count generation.

Another choice that could be improved was the decision to create self-made modules for simple operations, such as the unzipping of some files (**UNZIP**) or merging of gene counts into one table (**MERGY**). The inclusion of pre-made

modules developed in more detail would likely be more efficient and general but we valued the hands on experience of module development as more important and central to the objective of this project. This was a conscious decision to gain further practical experience with the Nextflow framework rather than solely using pre-implemented components. The simple scope of our custom modules was deliberately selected as tasks like file decompression and table merging are not computationally intensive, therefore even if unoptimized they do not pose as possible bottlenecks of the pipeline.

One more improvement could be the use of a second test dataset. We tried to implement one more test dataset in the `test_full.config`, where we aimed to use the example used in the nf-core pipeline `rnaseq`. The example is on the organism *Homo Sapiens*, but were not able to handle the data on our computers.

Our last uncertainty was the decision whether to use TPM (Transcripts Per Million) or raw gene counts to create the final merged table. We decided to calculate both, so if desired either can be used, but picked TPM as our main output as it is an already normalized, more significant measure of gene expression.

4.4 Outlook

The current pipeline is a solid base to build upon but further operations could be added to make it more complete. The most obvious addition is to include some differential expression analysis as the next step in the pipeline as the gene counts are already computed. To achieve this, another column must be added to the samplesheet to identify the conditions of which the expressions are then compared. A simple volcano plot which highlights the most significant and differential genes could be included.

References

- [1] Philip A. Ewels, Alexander Peltzer, Sven Fillinger, Harshil Patel, Johannes Alneberg, Andreas Wilm, Maxime Ulysse Garcia, Paolo Di Tommaso, Sven Nahnsen, et al. The nf-core framework for communitycurated bioinformatics pipelines. *Nature Biotechnology*, 38(3):276–278, 2020. doi: 10.1038/s41587-020-0439-x.
- [2] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, et al. The fair guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.
- [3] Nf-core docs: Getting started — nf-core. https://nf-co.re/docs/usage/getting_started/introduction, . Accessed: 2025-10-10.
- [4] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017. doi: 10.1038/nbt.3820.
- [5] Kimberly Robasky, Noreen E. Lewis, and George M. Church. Counting replicates counts: analysis of differential expression methods for rna-seq. *BMC Bioinformatics*, 13(1):184, 2012. doi: 10.1186/1471-2105-13-184.
- [6] Nf-core docs: Pipeline file structure. https://nf-co.re/docs/contributing/pipelines/pipeline_file_structure, . Accessed: 2025-10-18.
- [7] Nf-core modules. <https://nf-co.re/modules/>. Accessed: 2025-10-20.
- [8] Github repository: Trim galore! module. <https://github.com/nf-core/modules/tree/master/modules/nf-core/trimgalore>, . Accessed: 2025-10-21.
- [9] Github repository: Fastqc module. <https://github.com/nf-core/modules/tree/master/modules/nf-core/fastqc>, . Accessed: 2025-10-20.
- [10] Github repository: Star genomegenerate module. <https://github.com/nf-core/modules/tree/master/modules/nf-core/star/genomegenerate>, . Accessed: 2025-10-21.
- [11] Github repository: Star align module. <https://github.com/nf-core/modules/tree/master/modules/nf-core/star/align>, . Accessed: 2025-10-21.
- [12] Github repository: module. <https://github.com/nf-core/modules/tree/master/modules/nf-core/custom/getchromsizes>, . Accessed: 2025-10-21.
- [13] Github repository: Samtools sort. <https://github.com/nf-core/modules/tree/master/modules/nf-core/samtools/sort>, . Accessed: 2025-10-21.
- [14] Github repository: Picard markduplicates module. <https://github.com/nf-core/modules/tree/master/modules/nf-core/picard/markduplicates>, . Accessed: 2025-10-21.

- [15] Github repository: Subread featruecounts. <https://github.com/nf-core/modules/tree/master/modules/nf-core/subread/featurecounts>, . Accessed: 2025-10-21.
- [16] Center for Computational Biology Johns Hopkins University. Stringtie - transcript assembly and quantification for rna-seq. <https://ccb.jhu.edu/software/stringtie/>. Accessed: 2025-10-21.
- [17] Github repository: Stringtie_stringtie. <https://github.com/nf-core/modules/tree/master/modules/nf-core/stringtie/stringtie>, . Accessed: 2025-10-21.
- [18] Github repository: Stringtie_merge. <https://github.com/nf-core/modules/tree/master/modules/nf-core/stringtie/merge>, . Accessed: 2025-10-21.
- [19] Nf-core docs: Create a module. <https://nf-co.re/docs/nf-core-tools/modules/create>, . Accessed: 2025-10-21.